



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)



AI-Based Email Spam Predictor Using Machine Learning

(VITHARTHI)

REPORT

2025–26

SUBMITTED BY: Manan Sharma

B.Tech Computer science engineering (Core)

Registration Number: 25BCE11179

VIT Bhopal

INTRODUCTION

This project is a simple, practical, and efficient **AI-based Email Spam Predictor** that helps users identify whether an email is **Spam** or **Not Spam**. In today's digital era, email has become one of the most important modes of communication in personal, academic, and professional environments. However, the increasing number of spam emails, promotional messages, phishing attempts, and fraudulent content creates major challenges such as wasted time, reduced productivity, and potential security risks.

This project provides an easy-to-use, browser-based solution by combining **Machine Learning, Natural Language Processing (NLP)**, and an interactive **Streamlit web interface**. The system analyzes the content of an email and automatically classifies it as spam or not spam, helping users manage their inbox more efficiently and safely.

The application offers multiple functionalities, including:

- manual email input for instant prediction
- CSV file upload for bulk email analysis
- dataset upload for retraining the model dynamically
- downloadable results for further use

The project is developed using **Python, Streamlit, Pandas, Count Vectorizer, and Multinomial Naive Bayes**. The text data is first converted into numerical form using Count Vectorizer, and then the Naive Bayes algorithm is applied to perform classification based on word patterns and frequencies.

A key advantage of this system is its simplicity and real-world usefulness. It is designed to be lightweight, fast, and beginner-friendly, making it accessible even to non-technical users. At the same time, it provides a strong understanding of how Machine Learning models work in real-world applications.

This project is especially useful for **students learning AI and NLP, developers building machine learning applications, educators demonstrating practical concepts, and general users who want a quick spam detection tool**.

Overall, this project demonstrates how Machine Learning can be effectively used to automate email filtering and improve communication security in a simple and efficient way.

PROBLEM STATEMENT

Email communication is one of the most widely used methods for sharing information in personal, academic, and professional environments. However, with the growing use of email services, the number of **spam, promotional, phishing, and scam emails** has also increased significantly. These unwanted messages often create problems such as **inbox clutter, wasted time, reduced productivity, and potential cybersecurity threats**.

Many users still depend on manually checking emails, which is both time-consuming and inefficient. In some cases, harmful phishing emails may look similar to genuine messages, making it difficult for users to identify them correctly. This increases the risk of clicking unsafe links, downloading malicious attachments, or sharing sensitive information.

Existing email services do provide spam filters, but they are often limited, difficult to customize, or not suitable for educational demonstration purposes. Therefore, there is a need for a **simple, intelligent, and browser-based spam detection system** that can automatically analyze email text and classify it as **Spam** or **Not Spam** in real time.

This project addresses that need by using **Machine Learning and Natural Language Processing techniques** to build an efficient spam prediction system. The aim is to provide users with a fast, accurate, and easy-to-use solution for filtering unwanted emails while also demonstrating the practical application of AI in solving real-world communication problems.

FUNCTIONAL REQUIREMENTS

The system should provide the following main functions based on the implemented features:

1. Manual Email Text Input

The application should allow the user to type or paste an email message into a text box.

After clicking the **Predict** button, the system should analyze the entered text and classify it as:

- **Spam Email**
- **Not Spam**

This feature helps users quickly check a single email message.

2. CSV File Upload

The system should allow the user to upload a **CSV file** for checking multiple emails at once.

The uploaded CSV file must contain a **text column** that stores all email messages.

This feature is useful for bulk spam prediction.

3. CSV Data Preview

After uploading the CSV file, the system should display the first few rows of the uploaded data.

This allows the user to confirm that the correct file has been uploaded before prediction.

4. Bulk Email Prediction

After clicking the **Predict CSV** button, the system should:

- read all email messages from the **text column**

- convert them into numerical format
- predict each message
- add a new **Prediction column**

The prediction result should clearly show:

- **Spam**
- **Not Spam**

5. Download Prediction Results

The system should provide a **Download Results** button after bulk prediction.

This allows the user to download the updated CSV file with prediction labels for future use or record keeping.

6. Warning and Error Messages

The application should display proper messages for:

- empty text input
- missing **text column**
- invalid CSV structure
- successful prediction output

This makes the application easy to use and understandable for all users.

NON-FUNCTIONAL REQUIREMENTS

The system should satisfy the following non-functional requirements to ensure smooth performance and a good user experience:

1. Performance

The application should provide quick predictions for both single email input and CSV file upload.

The response time should be fast so that users can get results without unnecessary delay.

2. Usability

The interface should be simple, clean, and easy to understand.

Even users with basic computer knowledge should be able to use the system without difficulty.

3. Reliability

The system should give consistent and correct results for the same email input every time it is tested.

It should also work properly for repeated CSV uploads.

4. Compatibility

The application should run smoothly on all major web browsers such as:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox

This makes the project easy to access on different systems.

5. Maintainability

The code should remain well-structured and easy to update.

New features such as better machine learning models or additional file formats should be easy to add in the future.

6. Scalability

The system should be able to handle larger CSV files with many email messages without affecting performance significantly.

This helps the project remain useful for bigger datasets.

7. Portability

The application should run on different devices and operating systems where Python and **Streamlit** are supported.

It can be executed on local machines as well as cloud platforms.

SYSTEM ARCHITECTURE

The Email Spam Predictor follows a simple and efficient architecture that combines a **user-friendly frontend interface** with a **machine learning backend model**.

The overall system is divided into the following main parts:

1. User Interface Layer

This layer is built using **Streamlit**.

It provides all the components through which the user interacts with the system, such as:

- email text input box
- CSV file uploader
- prediction buttons
- result display section
- download button

This layer is responsible for collecting user input and showing the final prediction output.

2. Data Processing Layer

Once the user provides text input or uploads a CSV file, the data moves to the processing layer.

In this step:

- the email text is extracted
- unwanted formatting is removed if needed
- the text is prepared for machine learning prediction

This layer ensures that the input is in the correct format before sending it to the model.

3. Text Vectorization Layer

The prepared text is converted into numerical form using **CountVectorizer**.

This converts words into word-frequency vectors, which helps the machine learning model understand the text data.

This is an important NLP step in the system.

4. Prediction Model Layer

The numerical vectors are passed into the **Multinomial Naive Bayes model**.

This layer performs the main task of classification and predicts whether the email is:

- **Spam**
- **Not Spam**

The prediction is based on the learned word patterns from the training dataset.

5. Output and File Handling Layer

The final prediction result is shown to the user through the Streamlit interface.

For CSV uploads, the system also:

- adds a **Prediction column**

- shows the updated table
- provides a **Download Results button**

This layer handles result presentation and file export.

Overall Workflow

User Input → Data Processing → Text Vectorization → Spam Prediction → Display Result / Download CSV

DESIGN DECISIONS & RATIONALE

The overall design of this project is based on the idea of **simplicity, speed, and practical usability**. The aim was to create a spam prediction system that is easy to understand, quick to run, and useful for academic as well as real-world purposes.

Streamlit was chosen for the frontend because it allows fast development of interactive browser-based applications with minimal code. It provides a clean interface for users to enter email text, upload CSV files, and view prediction results in a simple way.

For text processing, **CountVectorizer** was selected because it efficiently converts raw email text into numerical **word-frequency vectors**. Since machine learning models cannot directly understand text, this step is necessary to transform the input into a format suitable for prediction.

The prediction model used is **Multinomial Naive Bayes**, which is one of the most effective algorithms for text classification tasks. It was selected because it is:

- fast in prediction
- lightweight in memory usage
- easy to implement
- highly suitable for spam detection
- widely used in real-world NLP applications

These design choices help make the system reliable, beginner-friendly, and academically strong while maintaining good prediction performance.

IMPLEMENTATION DETAILS

The system starts by loading a **default pre-trained spam prediction model** when the Streamlit application runs. This model is trained on a small sample dataset containing spam and non-spam email examples.

For Manual Email Input

When the user types an email message:

1. the input text is captured from the Streamlit text area
2. the text is passed to **CountVectorizer** for conversion into numerical vectors
3. the processed vector is sent to the **Naive Bayes model**
4. the model predicts whether the email is spam or not spam
5. the result is displayed on the screen using success or warning messages

This feature is mainly useful for quick checking of a single email.

For CSV Upload

When the user uploads a CSV file:

1. the file is read using **Pandas**
2. the system checks whether the file contains a valid **text column**
3. all rows from the text column are extracted
4. the text data is vectorized
5. predictions are generated for each email row
6. a new **Prediction column** is added to the dataset
7. the updated table is displayed on the screen

- the user can download the final CSV file with results

This feature is useful for **bulk spam prediction and dataset analysis**.

For Future Dataset Upload Enhancement

If the training dataset upload feature is added in future:

- the dataset file will be uploaded
- required **text and label columns** will be validated
- CountVectorizer will be rebuilt
- the model will be retrained on the new data
- the updated model will be stored in session state for further predictions

This will make the project more flexible and suitable for advanced learning.

SCREENSHOTS / RESULTS

```
email_class.py / ...
1 import streamlit as st
2 import pandas as pd
3
4 from sklearn.feature_extraction.text import CountVectorizer
5 from sklearn.naive_bayes import MultinomialNB
6 # Model train
7 @st.cache_resource
8 def train_model():
9     data = {
10         "text": [
11             "Win money now",
12             "Free prize claim now",
13             "Limited offer click here",
14             "Congratulations you won lottery",
15             "Claim your free gift card now",
16             "Earn money fast from home",
17             "Hello friend how are you",
18             "Let's meet tomorrow",
19             "Project meeting schedule",
20             "Lunch at 2pm",
21             "Can we reschedule the meeting",
22             "Please review the document"
23         ],
24         "label": [1,1,1,1,1,1,0,0,0,0,0,0]
25     }
26
27     df = pd.DataFrame(data)
28
29     vectorizer = CountVectorizer()
30     X = vectorizer.fit_transform(df["text"])
31     y = df["label"]
32
33     model = MultinomialNB()
34     model.fit(X, y)
35
36     return model, vectorizer
```

```
39 model, vectorizer = train_model()
40 # Streamlit UI
41 st.set_page_config(page_title="Spam Detector", page_icon="📧")
42
43 st.title("Email Spam Detector")
44 st.markdown("Detect whether an email is **Spam or Not Spam**")
45
46 option = st.radio("Choose input method:", ["Type Email", "Upload CSV"])
47
48 # text input
49 if option == "Type Email":
50     user_input = st.text_area("Enter your email text:")
51
52     if st.button("Predict"):
53         if user_input.strip() != "":
54             transformed = vectorizer.transform([user_input])
55             prediction = model.predict(transformed)[0]
56
57             if prediction == 1:
58                 st.error("Spam Email")
59             else:
60                 st.success("Not Spam")
61         else:
62             st.warning("Please enter some text")
63
64 # CSV Upload
65
66 else:
67     uploaded_file = st.file_uploader("Upload CSV file", type=["csv"])
68
69     if uploaded_file is not None:
70         df = pd.read_csv(uploaded_file)
71
72         st.write("### Uploaded Data")
73         st.dataframe(df.head())
```

```
75 # Support both column names
76 if "text" in df.columns:
77     text_column = "text"
78 elif "email_text" in df.columns:
79     text_column = "email_text"
80 else:
81     st.error("CSV must contain 'text' or 'email_text' column")
82     st.stop()
83
84 if st.button("Predict CSV"):
85     transformed = vectorizer.transform(df[text_column].astype(str))
86     predictions = model.predict(transformed)
87
88     df["Prediction"] = ["Spam" if p == 1 else "Not Spam" for p in predictions]
89
90     st.write("### Results")
91     st.dataframe(df)
92
93     csv = df.to_csv(index=False).encode('utf-8')
94     st.download_button(
95         "Download Results",
96         csv,
97         "results.csv",
98         "text/csv"
99     )
```

Email Spam Detector

Choose input method:

- ☒ Type Email
☐ Upload CSV

Enter your email text:

Predict

Email Spam Detector

Choose input method:

- ☒ Type Email
☐ Upload CSV

Enter your email text:

Subject: 🎉 Congratulations! You've Won a Free Gift Card
Dear User,
We are excited to inform you that you have been selected as the lucky winner of a **₹50,000 Amazon Gift Card**.

Predict

🚫 Spam Email

Email Spam Detector

Choose input method:

- ☐ Type Email
☒ Upload CSV

Upload CSV file

Drag and drop file here
Limit 200MB per file • CSV

Browse files

Email Spam Detector

Choose input method:

- ☐ Type Email
☒ Upload CSV

Upload CSV file

Drag and drop file here
Limit 200MB per file • CSV

Browse files

📄 spam_test_dataset.csv 0.8KB

Uploaded Data

	email_text	label
0	Congratulations! You've won a free iPhone. Click here now!	spam
1	Urgent! Your bank account has been compromised. Verify immediately.	spam
2	Win cash prizes up to \$10,000. Limited time offer!	spam
3	Get cheap meds without prescription. Order today!	spam
4	You have been selected for a lottery. Claim your reward now!	spam

TESTING APPROACH

The system was tested using different types of email inputs to ensure that the spam prediction model works correctly and gives reliable results.

1. Manual Testing

Single email messages were tested by typing text manually into the application.

Spam-related words and phrases such as:

- **win money**
- **free lottery**
- **claim reward**

were used to verify whether the model correctly identifies spam messages.

2. Genuine Email Testing

Normal day-to-day email messages were also tested, such as:

- **project meeting**
- **lunch tomorrow**
- **assignment deadline**

This was done to check whether the model correctly classifies genuine emails as **Not Spam**.

3. CSV File Testing

Multiple email messages were stored in a CSV file and uploaded to the application.

The prediction results for all rows were checked carefully to verify that the system correctly performs **bulk spam prediction**.

4. Future Dataset Retraining Testing

For future enhancement, custom datasets can be uploaded to retrain the model.

This testing ensures that the updated model can learn from new data and improve prediction flexibility.

CHALLENGES FACED

During the development of this project, a few practical challenges were faced.

Some of the main challenges included:

- handling proper **CSV file validation**
- checking whether the required **text column** exists
- managing UI refresh behavior in Streamlit
- keeping the code structure clean and beginner-friendly
- ensuring stable prediction results for repeated inputs

These challenges helped improve both the coding structure and the usability of the final project.

Learnings & Key Takeaways

This project provided strong practical learning in both machine learning and web application development.

The main learnings from this project include:

- understanding the basics of **Natural Language Processing**
- implementing **CountVectorizer** for text conversion
- applying **Naive Bayes for spam classification**
- building interactive applications using **Streamlit**
- handling CSV files with **Pandas**
- understanding how real-world spam filtering systems work

Overall, this project improved both technical knowledge and practical implementation skills.

FUTURE ENHANCEMENTS

The current system works well for basic spam prediction, but it can be improved further with advanced features.

Possible future enhancements include:

- using **TF-IDF vectorization**
- applying **Logistic Regression**
- using **Deep Learning NLP models**
- adding **accuracy score calculation**
- generating a **confusion matrix**
- adding **charts and analytics dashboard**
- deploying the app on **cloud platforms**

These improvements can make the project more advanced and suitable for large-scale real-world usage.

-REFERENCES-

The following resources were referred to during the development of this project:

- **Scikit-learn Documentation**
- **Streamlit Documentation**
- **Pandas Documentation**
- **Natural Language Processing blogs and tutorials**
- **YouTube implementation guides**
- **Machine Learning project references**

These resources helped in understanding the algorithms, libraries, and implementation workflow used in the project.